



FRESH FOOD • BETTER LIFE

ABC FOODMART

Relational Database & Dashboard System

APAN 5310 · SQL & Relational Databases · Final Presentation
Group 4: Xiangzhou Tan, Xilin Wang, Guyi Jia, Cloris He

PostgreSQL

Python ETL

Metabase

1. Client Scenario: Why ABC Foodmart Needs a New System

Consulting problem definition — business growth has outpaced the spreadsheet-and-binder workflow.

SITUATION

ABC Foodmart is expanding from two Queens locations to three new Brooklyn stores. Staffing, inventory, purchasing, sales, and accounting data must now scale across five locations.

COMPLICATION

Operational information is fragmented across spreadsheets and paper records. Decisions are slow, cross-store comparisons are difficult, and inconsistent updates increase the risk of errors.

OUR RESPONSE

Build one normalized PostgreSQL database, automate data preparation with Python, and expose role-appropriate insights through SQL and Metabase dashboards.

CLIENT
DEMAND

01

Faster decisions

inventory, purchasing, staffing

02

Reliable reporting

single source of truth

03

Scalable control

five stores + future growth

2. Data Structure – “Original Data”

Because no usable historical source file was provided, we modeled representative raw records the new system would receive.

VALIDATION SUMMARY

```
Authoritative final PostgreSQL
row counts
-----
stores = 5
job_roles = 6
employees = 80
staff_shifts = 3627
time_off_requests = 80
product_categories = 8
products = 48
store_inventory = 126
inventory_adjustments = 128381
vendors = 10
vendor_products = 93
purchase_orders = 1240
purchase_order_items = 4943
deliveries = 1210
sales_transactions = 36000
sales_items = 122906
customer_returns = 720
payments = 37333
operating_expenses = 119
```

DATA STAGING

```
customer_returns.csv
deliveries.csv
employees.csv
inventory_adjustments.csv
job_roles.csv
operating_expenses.csv
payments.csv
product_categories.csv
products.csv
purchase_order_items.csv
purchase_orders.csv
sales_items.csv
sales_transactions.csv
staff_shifts.csv
store_inventory.csv
stores.csv
time_off_requests.csv
vendor_products.csv
vendors.csv
```

Dataset note

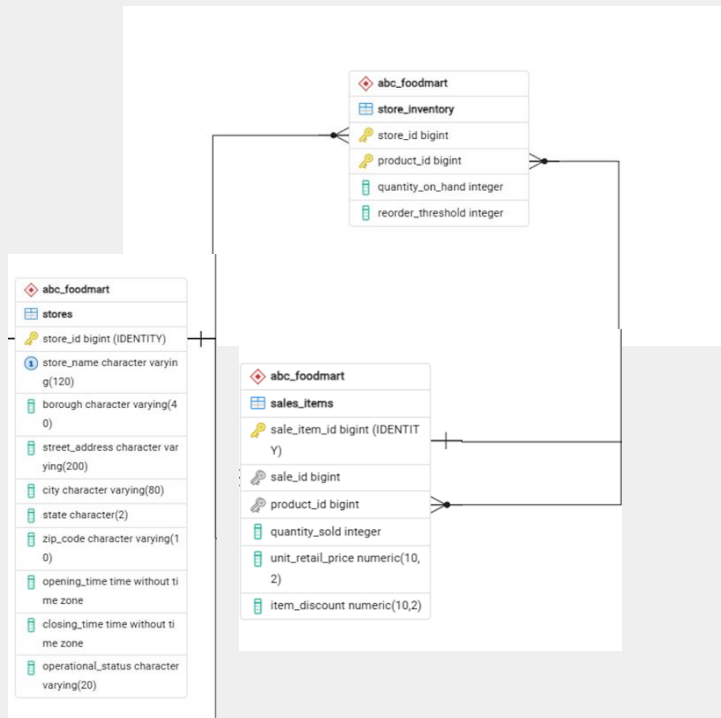
The package contains exactly 19 CSV files. The staged `inventory_adjustments.csv` contains 5,475 non-sale adjustments. During PostgreSQL loading, the validated sales-item trigger adds 122,906 sale adjustments, producing the authoritative loaded count of 128,381.

DATA GENERATE

ABC Foodmart did not provide us with a complete historical dataset, so we treated this as the design of a new operational database rather than a traditional data migration project. We first defined what data each business process should generate, and then created synthetic data for stores, employees, products, inventory, purchases, deliveries, sales, returns, and expenses.

Sample Schema: SQL Demo

Demo example



```
CREATE TABLE stores (
  store_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  store_name        VARCHAR(120) NOT NULL UNIQUE,
  borough           VARCHAR(40) NOT NULL CHECK (borough IN ('Queens', 'Brooklyn')),
  street_address    VARCHAR(200) NOT NULL,
  city              VARCHAR(80) NOT NULL DEFAULT 'New York',
  state             CHAR(2) NOT NULL DEFAULT 'NY',
  zip_code          VARCHAR(10) NOT NULL,
  opening_time      TIME NOT NULL,
  closing_time      TIME NOT NULL,
  operational_status VARCHAR(20) NOT NULL DEFAULT 'active'
    CHECK (operational_status IN ('planned', 'active', 'temporarily_closed', 'closed')),
  CHECK (opening_time < closing_time)
);

CREATE TABLE store_inventory (
  store_id          BIGINT NOT NULL REFERENCES stores(store_id),
  product_id        BIGINT NOT NULL REFERENCES products(product_id),
  quantity_on_hand  INTEGER NOT NULL DEFAULT 0 CHECK (quantity_on_hand >= 0),
  reorder_threshold INTEGER NOT NULL DEFAULT 0 CHECK (reorder_threshold >= 0),
  PRIMARY KEY (store_id, product_id)
);

CREATE TABLE sales_items (
  sale_item_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sale_id           BIGINT NOT NULL REFERENCES sales_transactions(sale_id) ON DELETE CASCADE,
  product_id        BIGINT NOT NULL REFERENCES products(product_id),
  quantity_sold     INTEGER NOT NULL CHECK (quantity_sold > 0),
  unit_retail_price NUMERIC(10,2) NOT NULL CHECK (unit_retail_price >= 0),
  item_discount     NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (item_discount >= 0),
  UNIQUE (sale_id, product_id),
  CHECK (item_discount <= quantity_sold * unit_retail_price)
);

CREATE TRIGGER trg_reduce_inventory_after_sale
AFTER INSERT ON sales_items
FOR EACH ROW EXECUTE FUNCTION reduce_inventory_after_sale();
```

Sample Data: SQL Demo

Demo example

```
SELECT s.store_name,  
       p.product_name,  
       i.quantity_on_hand,  
       i.reorder_threshold  
FROM store_inventory i  
JOIN stores s  
  ON s.store_id = i.store_id  
JOIN products p  
  ON p.product_id = i.product_id  
WHERE i.quantity_on_hand <= i.reorder_threshold  
ORDER BY s.store_name, i.quantity_on_hand;
```

RESULT

```
Flushing | Whole Milk | 7 on hand | threshold 10
```

WHY THIS MATTERS ?

01

Operational

joins inventory, store, and product context

02

Actionable

returns only items requiring replenishment attention

03

Reusable

same query powers analyst workflows and dashboard cards

4. Design Choices: Normalized, Connected, Enforced

We designed for maintainability first — then let SQL joins reconstruct the business view when needed.

01

3NF NORMALIZATION

Separate stores, employees, categories, vendors, orders, transactions, and expenses so each fact is stored once and update anomalies are minimized.

02

ASSOCIATIVE TABLES

Bridge many-to-many relationships such as vendor-product and sale-product. This preserves flexibility without duplicating descriptive attributes.

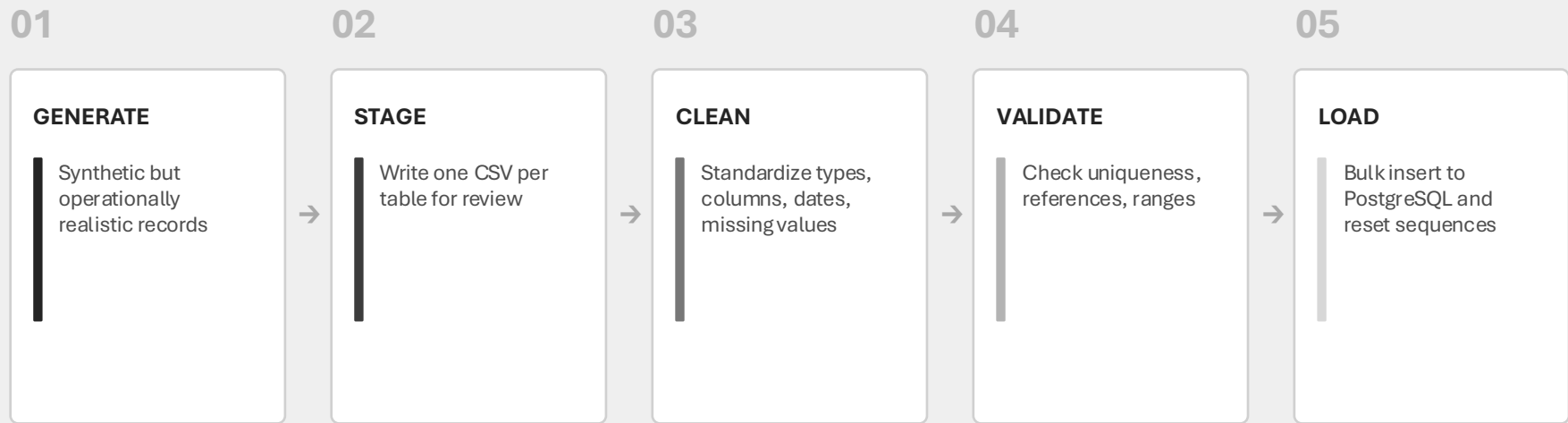
03

INTEGRITY RULES

Primary keys, foreign keys, CHECK constraints, and NOT NULL rules protect valid references and reduce bad operational states.

5. Data Pipeline & ETL: Python → Validation → PostgreSQL

The pipeline generates reproducible test data, standardizes it, validates relationships, and loads tables in dependency order



NOTE

Fixed random seed 5310 · transaction-safe load · final row-count checks · rollback on failure

6. Analytical Procedures: Customer Needs → SQL Questions

The database is useful because it converts management questions into repeatable, auditable analytical queries.

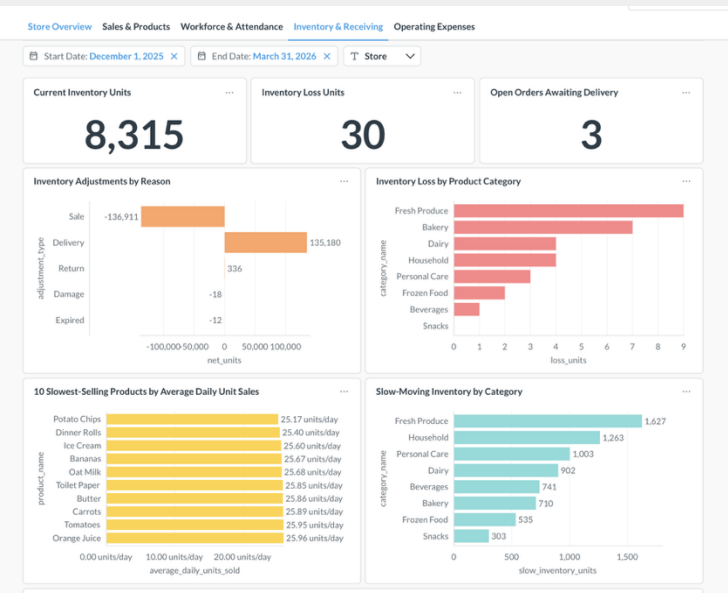
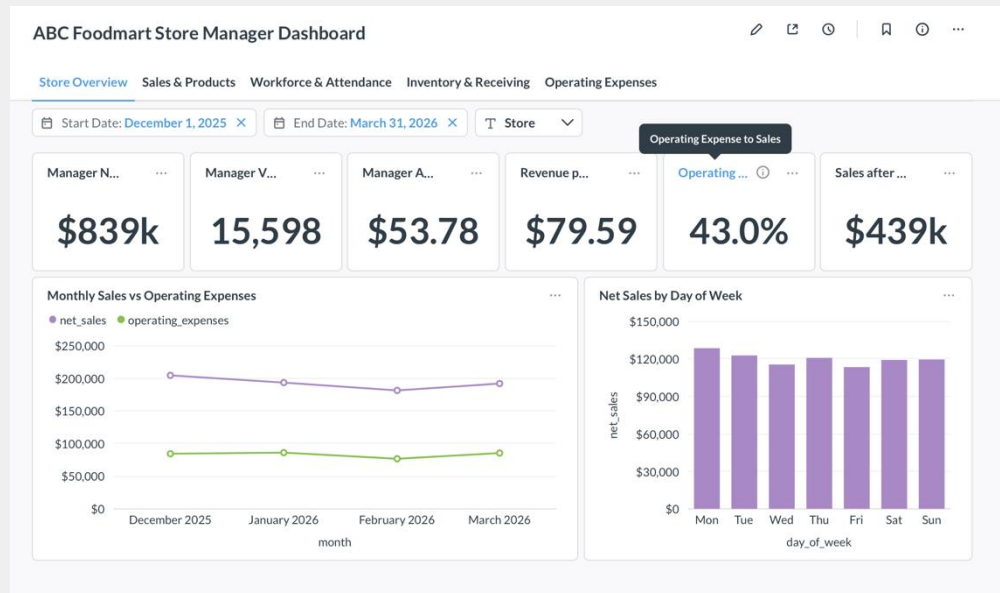
BUSINESS QUESTION	ANALYTICAL OUTPUT	RELATIONAL PATH
Stock risk	Which products are at risk of going out of stock?	store_inventory + products + stores
Overstock	Which items have high inventory but weak sales?	store_inventory + sales_items + sales_transactions
Product performance	Which products/categories sell best by store?	sales_transactions + sales_items + products + product_categories
Return Analysis	Which products and product categories have a high return rate?	customer_returns + sales_items + products + product_categories
Store profitability	Which stores underperform after operating costs?	sales_transactions + operating_expenses + staff_shifts + employees + stores
Staffing	Which stores have frequent lateness, absences, or staffing gaps?	staff_shifts + employees + time_off_requests + stores

ANALYST ACCESS raw SQL · Python/R connector · detailed drill-down

MANAGER ACCESS Metabase dashboards · filters · KPI overview

7. Metabase Demo: Management View

http://82.157.4.243:3000



DEMO FLOW Filter store → inspect low-stock items → drill into product/vendor details → return to executive KPIs